

What Scale Buys in Tool-Use Agents, and How to Buy It Cheaply: A Capability×Scale×Lever×Cost Map Where Compliance Is Scale-Invariant

Merged ICLR draft (Papers 1+4), branch `facet-rft-2026`. This paper is the realization of the project master plan `EXPERIMENT_DESIGN.md` §0★★/★★★ — the capability×scale×lever×cost map — combining the measured science (what scale buys; ex-Paper 1) with the cost economics (how to buy it cheaply; ex-Paper 4). Companion papers: A2-generation (Paper 2) and learned path-selection (Paper 3) remain separate. Numbers tagged [SETTLED] are measured with zero paid frontier-API calls (local user-simulator); [EST] are projections/planned. §7 is the funded forward plan (\$0.5–1k budget).

Abstract

Tool-use agents (NL request → tool calls under a policy) are usually improved by scaling the base model. We instead ask, function by function and across scale: **what does scale actually buy, what can no model do at any scale, what is covered cheaply by deterministic scaffold or authored per-domain relations (A2) — and what is the total-cost-optimal way to assemble these?** Treating four benchmarks (τ^2 -bench, SOPBench, TaskBench, Synth) as measurement instruments, we decompose tool-use failure into **operand execution**, **orchestration load**, and **compliance/guarantee**, chart each across scale (7B–32B measured; 72B–235B [EST]) and lever (base / capability-scaffold / guarantee-scaffold / A2 / learning), and fold the map into a **total-lifecycle cost objective** (CapEx + OpEx + maintenance + experts + per-field generalization). Findings: (1) **operand execution saturates early** — a 32B base is 88/88 (100%) exact given a specification, so it is not a learnable gap at 32B. (2) **Capability-type load retires with scale in a definite order** — of five candidate load dimensions, interference and state-multiplicity bind at 7B/14B but vanish by 32B, while length and conditional-depth persist; a controlled probe overturns the observational interference signal. (3) **Compliance is scale-invariant** — task-pass climbs with scale (7B 0.19→32B 0.55) but the *compliant*-pass gap stays flat (~ 0.056) and policy-semantic violations do not fall (g2 31→41 across 7B→32B) even as auth-type violations collapse (g1 57→6); a deterministic gate eliminates them at every scale. We frame deterministic scaffold as a **load-reduction operator** ($\text{pass} \Leftrightarrow L_{\text{eff}} < L^*(N)$, $L_{\text{eff}} = L - \Delta L_{\text{scaffold}}$), and we measure a deployment payoff: on τ^2 -retail, a 32B-int8 base + deterministic gate runs on-prem at $\approx 23\times$ lower \$/req than a frontier API at near-equal compliance, and a small-on-prem + escalation **fleet** reaches equal-or-better compliance at $\sim 2.1\times$ lower cost [SETTLED on existing data; router decidable, projected]. We conclude that **scale buys capability but not guarantee**, so even a frontier model needs a deterministic guarantee-layer, and the cost-optimal object is not the smallest model but the **total-cost knee**: a small base + a capability-scaffold that recedes with scale + a scale-invariant guarantee-scaffold + authored A2, with learning reserved for a small, scale-dependent residual.

1. Introduction

A tool-use agent maps a natural-language request to a sequence of tool calls satisfying a domain policy. The default recipe is a bigger base model — but "bigger" conflates distinct capabilities and is expensive on **two permanent axes**: inference OpEx (a $2\times$ model is $\sim 2\times$ \$/req and slower, forever) and, for on-prem/sovereign deployment, hardware CapEx (the operator *buys* the VRAM/power/cooling). The practical question is therefore not *how big a model* but **which capabilities must scale, which can be handed to cheap deterministic structure, which no model can be trusted to do, and what total-cost mixture is optimal**.

We answer with a **capability × scale × lever × cost** decomposition — the master map of `EXPERIMENT_DESIGN`. For each functional sub-capability we ask, across scale, whether it (a) grows with scale, (b) is a genuine model limit, (c) is covered by deterministic *scaffold*, or (d) is covered by per-domain authored relations (A2); we then assemble the per-capability levers under a total-lifecycle cost objective and locate the cost knee. Benchmarks are instruments for filling cells, not the object of study.

Objective function (the economics, ex-Paper 4). We minimize, *simultaneously*,

```
{ ① cheap GPU tier/count · ② low VRAM (int8/int4) · ③ maintenance per change · ④ human experts · ⑤ generalization cost per new field (ideally A2-swap only) } + HW CapEx + inference OpEx. The terms are coupled and point the same way: ①② → a small model + quantization; ③④⑤ → maximize the fixed domain-general part (deterministic scaffold + transferable, forgetting-free learning = one-time) and minimize the per-domain A2.
```

The **common enemy is the A2** (it drives ③④⑤ and is worst on-prem, where data cannot leave so a frontier A2-generator is unavailable); the **common friend is fixed scaffold + transferable TBox**. The optimum is the **total-cost knee**, not the smallest model.

Contributions. 1. A controlled, **frontier-API-free** methodology isolating operand / orchestration-load / compliance across scale (§4): isolated single-shot probes + existing multi-scale runs + a local user-simulator avoid the cost and noise of paid full rollouts. 2. **Operand execution is scale-saturated, not learnable at 32B** (§5.1); the apparent gap is criterion-interpretation (deterministic compute) and join/disambiguation (present-addressable). Faithful-formalize SFT is **NO-GO at 32B** — but possibly **GO at 7B**, where a residual exists (§7). 3. A **load decomposition with a scale-response** (§5.2): the binding load set is *scale-dependent*; scale retires interference/state before length/conditional. Controlled generation overturns an observational correlation — the methodological payoff. 4. **Compliance/guarantee is scale-invariant** (§5.3): capability rises with scale but policy-semantic violation does not; a deterministic gate closes it at every scale. This is the empirical backbone of a **two-kind scaffold** — capability-scaffold recedes with scale (F3), guarantee-scaffold is scale-invariant (F4) — and explains *why even frontier systems need a deterministic guarantee layer*. 5. A **total-cost-optimal lever allocation** with a measured deployment payoff (§5.5–6): decidability-first routing, $\approx 23\times$ \$/req TCO advantage and a $2.1\times$ fleet, and the cost-knee framing — the engineering realization of the map. 6. A unifying frame: **scaffold = a load-reduction operator** (§3), $\text{pass} \Leftrightarrow L_{\text{eff}} < L^*(N)$, with an *independent* estimate of ΔL (we establish the components; the full predictive test is §7 [EST]).

Our headline is deliberately **not** "small beats large" (held by RL-routing systems, §2). It is *which* parts of the gap are scale-invariant and must be offloaded, and the cost-optimal map telling a deployer which lever to pull per capability.

2. Related Work

The contribution sits at an intersection of verified prior pieces; the central lineage is conceptual (why an LLM is good at language but must offload deep procedures and cannot self-guarantee compliance), not the recent "small-beats-large" routers (§2.5).

§2.1 Language is shallow-parallel; deep procedures must be offloaded. A bounded-depth Transformer forward pass is asymptotically uniform TC■ (Merrill & Sabharwal, 2207.00729); CoT escapes only by serializing depth (Feng et al., 2305.15408/2402.12875). Natural language embeds procedures of definite class — van Benthem's semantic automata ("most" is pushdown), cognitively real (working-memory/PFC scaling); Relevance Theory's procedural/conceptual split; Goodman notationality (cost not computability, per Zhang & Norman). So *classifying* which deep procedure a request embeds is shallow and learnable; *executing* it is serial-deep and offloaded — the basis of capability/lever routing.

§2.2 Cognitive architectures, non-introspective decidability, scale-invariant epistemic limits. Our system instantiates the CoALA umbrella (Sumers et al., 2309.02427, TMLR), making concrete what it leaves general (a relational-algebra deterministic core; empty-relation abstention; zero-retrain A2-swap). SOAR (Newell/Laird/Rosenbloom 1987; Laird 2012; Wray/Kirk/Laird, 2505.07087) surfaces indecision *architecturally* (an *impasse* from absent preference, not introspection) — the precedent for "uncertainty $\rightarrow$ observable empty/ambiguous relation $\rightarrow$ ASK." Deterministic-first/LLM-fallback is established (Zhu & Simmons, 2403.00810, 50–100 $\times$ fewer tokens; NL2GenSym, 2510.09355). We add a **formalize-faithfulness** axis (SOAR assumes productions correct), a *learned* empty $\rightarrow$ ASK, and a small weight-learned domain-general skill with A2-swap. On epistemics, hallucination is provably scale-invariant (Xu et al. 2401.11817; Kalai et al. 2509.04664: scoring rewards guessing over abstention); **our §5.3 is the policy-enforcement analogue** — no model self-guarantees compliance, just as none self-guarantees truthfulness; both demand an external deterministic mechanism.

§2.3 LLM-proposes / deterministic-executes. LLM-Modulo (2402.01817), PAL, NL $\rightarrow$ PDDL formalizers, RLVR with deterministic checkers (AlphaGeometry/Proof; Tülu 3, 2411.15124). Honest boundary: offload does not always win (formalization overhead on simple cases; spec leakage) — the boundary is itself measured.

§2.4 Guarantees / shielding. Safe-RL shielding (Alshiekh et al. 2018; Jansen et al. 1807.06096), predictive safety filters (Wabersich & Zeilinger 2021); LLM-agent instances ShieldAgent (2503.22738), AgentSpec (2503.18666), Formal-LLM (2402.00798). These veto with hand/probabilistic specs; our gate is deterministic, *drives* (repairs) preconditions, and pairs with a learned proposer and zero-retrain transfer.

§2.5 Learned routing / "small-model orchestration" (the narrow rivals). Routing is a learned transferable skill (RTR, A2FM, xRouter, ARM2, When2Call — tool-necessity linearly decodable, AUROC 0.89–0.96; RITE cross-domain). **ToolOrchestra (2511.21689)** RL-trains an 8B router to delegate to a frontier model (80.2%@10.3¢ vs GPT-5 77.7%@31.3¢ on τ^2) — strongest cost result, **but relies on frontier delegation**, disqualifying for the on-prem/no-egress regime; it optimizes cost *given* black-box

components and provides no deterministic offload, by-construction compliance, decidable-vs-learned accounting, or zero-retrain transfer. **TRUST (2606.06976)**: 4B>30B by RL, black-box, no guarantee. **ReDAct (2604.07036)** / Unified Routing-Cascading (2410.10347): calibrated deferral; our fleet is a *deterministic-router* instance. These establish that a cost-favorable small-model result *exists* — which is why our contribution is the *analysis and method* (what is scale-invariant; which lever per capability; the cost knee), not that result.

§2.6 Cost / regulatory / industrial. Against a pure-scale reading of the Bitter Lesson, Brooks' "A Better Lesson" (2019) and Chollet (1911.01547) argue total cost / priors must be counted — our knee operationalizes this. Buyer value of determinism = verifiability / model-risk exemption (SR 26-2; EU AI Act 2024/1689; EU MDR Annex I §17.1 repeatability). **Palantir Foundry** (Ontology $\approx$ scaffold+A2; AIP-LLM $\approx$ small model + transferable TBox) is the industrial baseline; we differentiate on *cost* (HW, experts, config-only field-crossing, zero egress), **not functionality**.

§2.7 Load / benchmarks. Load framing defers to cognitive-load theory and long-context "lost-in-the-middle"; novelty is only *load-decomposition* $\times$ *scaffold-load-reduction for tool-use*. Benches as instruments: τ^2 (2406.12045), SOPBench (2503.08669), TaskBench, Synth/CFB (2501.10132); floor-measurement precedent: "Sufficient Context" (Joren et al., ICLR 2025).

Whitespace. Prior work supplies each piece (procedural-semantics/automata, Transformer-expressivity, SOAR/CoALA, hallucination-inevitability, formalize-offload, learned routing, shielding, cost-routing). The unoccupied intersection is the **empirical capability $\times$ scale $\times$ lever $\times$ cost map**: which capabilities scale retires, which are genuine scale-invariant limits (compliance, measured), which lever covers each, and the **total-cost-optimal assembly**. Recent routers optimize cost over black boxes; we open the box.

3. The Capability $\times$ Scale $\times$ Lever $\times$ Cost Framework

We decompose a tool-use trajectory into typed functional steps; for each function f we populate a row indexed by scale N , lever $\blacksquare \in \{\text{base, capability-scaffold, guarantee-scaffold, A2, learning}\}$, and **assembled cost**.

Three structural claims. - **Operand vs orchestration vs compliance** fail for different reasons and respond to different levers: operand = atomic execution given a criterion; orchestration = holding interdependent steps coherent under *load*; compliance = obeying policy with a *guarantee*. - **Decidability-first lever routing** (cost-ascending short-circuit): *decidable* from schema+policy+state $\rightarrow$ scaffold; else domain fact $\rightarrow$ A2; else promptable at size $\rightarrow$ prompt; else scale-emergent & affordable $\rightarrow$ scale or cheap-replication; else irreducible NL $\rightarrow$ formalize $\rightarrow$ minimal transferable LoRA (last resort). This inverts "scale it." - **Two kinds of scaffold.** *Capability-scaffold* (present/compute/resolve) substitutes for a missing skill and **recedes as scale grows** (F3, raw pass). *Guarantee-scaffold* (policy gates) supplies a property no probabilistic model gives and is **scale-invariant** (F4, compliant-pass). §5.3 measures the split — it is *why even frontier keeps a deterministic trust layer*.

Load as a vector. Orchestration "load" is not scalar. From failure forensics: **L_len** (context to retain), **L_state** (interdependent state), **L_branch** (conditional depth), **L_intf** (confusable entities), **L_contra** (mid-stream revision). Crucially **load $\perp$ operand difficulty** (raise load with operand fixed).

Scaffold as a load-reduction operator. Each scaffold removes a dimension: present/autofetch $\downarrow$ L_len, $\downarrow$ L_intf; compute $\downarrow$ L_state; gates $\downarrow$ L_branch; controller $\downarrow$ L_state, $\downarrow$ L_len. This gives **L_eff** = **L** - Δ L_scaffold and the prediction pass(f, N) $\Leftrightarrow$ **L_eff**(f) < **L***(N), where **L***(N) is load-tolerance at scale N . To avoid tautology, Δ L is estimated *independently* (from the mechanical feature-reduction the scaffold performs) and shown to predict the failure-onset shift — not read off it. We establish the components here; the full predictive curve at >32B is §7 **[EST]**.

4. Method

Benches as instruments. τ^2 -retail (primary); SOPBench/TaskBench/Synth are the canonical *training* substrate for Papers 2–3 and the lever-allocation; τ^2 is a *transfer* target, never trained on. Scale: Qwen2.5 7B / 14B / 32B-GPTQ-Int8 **[SETTLED]**; 72B / 235B **[EST]** (§7).

Cost discipline (frontier-API-free). The only paid component is a GPT-4.1 user-simulator; we avoid it for discovery. Capability is measured with **isolated single-shot probes** and **existing multi-scale runs on disk**; a **local user-simulator** (faithful scripted turns over a local base model) replaces the paid sim for full-flow checks. Paid full-runs are used only as a **final confirmation**,

minimal scope, after a free conclusion (the budget posture of §7). All results persist under `sim_results/`.

Conditions. *floor* (no scaffold); *g15 / present / nested / assembled* (capability- and guarantee-scaffold); *GIVEN-SPEC* vs *GOAL* (separates operand execution from criterion interpretation).

Metrics. Robust **pass^{all}** over per-trial point estimates (user-sim noise ≈ 0.11 , so single-trial is not reported as a finding); **F3** = raw task pass; **F4** = compliant pass (deducting policy violations); per-dimension **partial correlation** of failure with each load feature (controlling operand difficulty); controlled **accuracy-vs-load** onset curves; and the **cost row** per cell (latency, tool-roundtrips, VRAM/GPU tier, \$/req).

5. Results

5.1 Operand execution is scale-saturated, not a learnable gap at 32B [SETTLED]

Varying only the user-sim input: given an explicit option spec (**GIVEN-SPEC**) the 32B base selects the correct variant **88/88 (100%)**; given only the goal (**GOAL**) it is 62/88 (70%), the 30% gap being criterion interpretation (argmax/argmin = deterministic compute; multi-attribute reasoning; conversational fidelity), not execution. Genuine join/disambiguation (user describes, not names, the order) is 7/13 (54%) and present-addressable. **Implication:** operand is not a 32B capability gap, and faithful-formalize SFT is **NO-GO at 32B** — no residual to learn. (*Forensic note: every prior "operand failure" dissolved into a measurement artifact — a calc bug, a premature case, an ID-given probe artifact — until the controlled probe read 100%. The corollary that matters for cost: at 7B the base may be genuinely weak even given-spec, so the learning lever can re-acquire a job at small scale — §7.*)

A complementary limit: *selection over a large candidate set does not scale in-head* — `comparative@50 = 0.02`, `rank@50  $\approx$  0.30` even including a 235B model, while a deterministic engine is 1.00 **[SETTLED]**. "Pick the best among many" is offloaded at any scale.

5.2 Capability-type load retires with scale, in order [SETTLED 7B–32B]

Partial correlation of *floor* failure with each load feature (controlling operand difficulty):

dimension	7B	14B	32B
L_len (length)	+0.20	+0.33	+0.37
L_state	+0.24	+0.21	+0.06
L_branch (conditional)	+0.20	+0.13	+0.26
L_interf	+0.30	+0.20	+0.08
L_contra	+0.06	+0.13	+0.04

The **binding set is scale-dependent**: at 7B/14B four dimensions predict failure; by 32B only length and conditional-depth survive (interference/state retire first). `L_contra` is too sparse in τ^2 to fit. **Controlled > observational**: a controlled interference probe (fix operand; vary only N confusable variants) does *not* reproduce the strong 7B signal —

N	1	2	4	8	16
32B	1.00	1.00	1.00	0.97	0.87
7B	1.00	1.00	1.00	0.90	0.80

(0.80 vs 0.87 at N=16): the observational `L_interf` was largely a size confound. (*Caution: single-shot probes under-test inherently multi-turn dimensions `L_state/L_contra`; controlled multi-turn probes are §7 [EST].*) **The residual is orchestration, not operand**: re-running the robust fail-all set under faithful local turns, 2/10 are sim-noise flips and the dominant residual is *orchestration-under-load* (batching that violates a precondition; multi-order tracking dropping an order + fabricating an id; conditional sequencing; context overflow). An isolation **plan probe** (elicit the abstract plan, reads pre-supplied, grade structure only) shows correct structure in 6/10 — i.e. $\sim$ half the orchestration residual is execution-load (closable by deterministic

plan/execute separation) and ~half is in-isolation planning miss (the candidate for a learned path-selection lever, Paper 3). Assembled-stack ceiling = robust pass^all **0.402 [SETTLED]**.

5.3 Compliance is scale-invariant — the central finding [SETTLED 7B–32B]

Separating **F3** (task pass) from **F4** (compliant pass) on *floor* (no gate):

scale	F3	F4	gap	viol (g1 auth / g2 policy)	total viol%
7B	0.189	0.130	0.058	57 / 31	33.6%
14B	0.468	0.404	0.064	38 / 38	26.3%
32B	0.547	0.491	0.056	6 / 41	14.3%

The F3–F4 gap is flat (~0.056) and **policy-semantic violations (g2) do not fall (31→41)** even as auth-type (g1) collapses (57→6). With the deterministic gate (g15): F3=F4, gap = **0.000**, viol = **0.0%** at every scale (7B=14B=32B).

Interpretation. Scale solves *capability-type* (g1) but **not policy-semantic** (g2) compliance; the gate zeros it at every scale, frontier included. This is the policy-enforcement analogue of hallucination-inevitability (§2.2) and the backbone of the two-kind scaffold. **★Honest caveat (load-bearing — to settle before submission).** Two of the three "invariance" signals are confounded by completion rate: total viol% *falls* (33.6→14.3) and violation-among-passing (gap/F3) *falls* (0.31→0.10) with scale; only the **absolute** F3–F4 gap (~0.056) and the **g2 raw count** (31→41) are flat/rising — and a higher-pass model completes more trajectories, i.e. has more write-opportunities to violate. The defensible claim is therefore the **differential**: scale collapses g1 (57→6) but barely touches g2 — *policy-semantic compliance is the residual a gate must own*. **Pre-submission requirement: report g2 as a per-write-opportunity rate with CI** (free re-analysis); if the rate is flat the strong "scale-invariant" claim holds, otherwise we state the differential form. Range is 7B–32B; the 72B point (§7) extends it.

5.3b Cheap-replication: a capability-scaffold substitutes for scale [SETTLED]

Raw pass rises monotonically (pass^1 = 7B 0.24 / 14B 0.52 / 32B 0.60, n=342) — the "scale buys capability" curve. The map's point is that *each piece is obtainable more cheaply than scale*: a deterministic **fetch-first** engine (provenance-deny → producer call → inject real value) moves grounding errors 33→9 and roughly **doubles pass (0.14→0.264) with zero learning** — a capability-scaffold standing in for scale. Lever audit: auth/confirm/ownership/precondition gates (G1–G4) are genuine levers; an eligibility-*steering* gate (G5) has ~zero causal effect (the model does not learn to use guidance) and naive retry is counter-productive — populating the "capability-scaffold recedes / guarantee-scaffold invariant" split with measured cells.

5.4 The map, filled

function	scale-response	genuine limit?	efficient lever
operand execution	saturates by 32B (100% given-spec)	no (at 32B)	base; learning may re-acquire job at 7B (§7)
interference / state load	retire by 32B	no	capability-scaffold (present), or scale
length / conditional load	persist at 32B	partial	capability-scaffold (partial); conditional → controller / learn
large-N selection	flat at all scales	yes (in-head)	deterministic compute
policy compliance (g2)	scale-invariant (differential)	yes (guarantee)	guarantee-scaffold (gate)
domain facts	n/a	n/a	A2 / retrieval

5.5 Cost: the deployment payoff [SETTLED on existing data]

- **≈23× TCO advantage.** On τ^2 -retail with existing runs: 32B-int8 + deterministic gate, on-prem, **\$0.0019/req vs frontier-API \$0.044/req** ≈ 23× (16–40× over GPU \$0.2–0.5/hr), ~3000× cheaper than a human agent (~\$6/contact); on-prem, auditable, zero egress. Honest trade-offs: latency ~6× (178s vs 30.5s); pure-on-prem compliance (0.573) is *below* frontier (0.82) → headline is "near-compliance, ~23× cheaper," not "equal." (*\$/req estimated from token=chars/4 + assumed GPU utilization — order-of-magnitude robust; exact multiple pending the litellm token-capture fix, \$7.*)
- **Fleet (equal-or-better compliance, cheaper).** Easy→32B on-prem, hard→frontier escalate: blended compliance 0.860 (> pure frontier 0.816) at \$0.021/req (~2.1× cheaper) on clean data [**SETTLED arithmetic; assumes a cheap *decidable* router, not yet implemented**].

6. Cost-Optimal Lever Allocation and the Knee

The map yields a deployment rule that inverts "scale it": **reduce the load and supply the guarantee with structure; do not train the model to tolerate either.** Capability-scaffold removes the load dimensions scale would otherwise buy (and recedes when scale is available); guarantee-scaffold supplies the property scale never buys (kept at every scale). Per-capability allocation is decidability-first (§3); the **common enemy is the A2** (minimized; Paper 2 auto-generates it), the **common friend is fixed scaffold + transferable TBox**. The optimum is the **cost knee** — the smallest *total-cost* size — and, more generally, a **heterogeneous on-prem fleet** whose mix is set by per-capability learn-vs-engine crossover, not a static assumption. Where a load is irreducible by scaffold (conditional-depth / path-search is our candidate), the lever shifts to a *learned, transferable* heuristic (Paper 3), not a bigger model. Positioning vs **Palantir Foundry** is on cost (HW/experts/field-crossing/egress), not functionality.

7. Funded Forward Plan (advancing the master map; budget ≤ \$1k)

This paper fills the *measured* cells of EXPERIMENT_DESIGN's 8-item checklist (items 1–4, 6 partial, 7); the budget below fills the highest-value remaining cells. Discipline: free local verification first; each paid run is a **confirmation of a free conclusion**, minimal scope.

1. **[FREE] g2 per-opportunity rate + CI** — settles the §5.3 backbone (compliance invariance vs differential). *Prerequisite to submission.*
2. **[≈\$30–80] 72B compliance point** — extends F3/F4 invariance from ≤32B to ≤72B (eval-item 2/7); the single largest credibility gain for the central claim. One scale, gated arm, local user-sim where possible.
3. **[≈\$50–150] 7B/14B scale axis with the *same* scaffold** — eval-item 2/3 (scale plan, §0★★★★ AM): does the fixed scaffold+A2 carry small models, and where does the base become too weak so the **learning lever re-acquires a job** (operand/orchestration residual exists at 7B though not 32B)? This is the cost-knee's left edge and tests whether cheap-replication moves the knee toward 7B.
4. **[≈\$50–150] cheap-replication confirmations** — fetch-first / present / controller-vs-learn on the orchestration residual (eval-item 3): confirm scaffold ΔL on a clean robust measure; estimate ΔL independently and test the load-reduction prediction (§3) at ≥2 scales.
5. **[≈\$80–200] multi-field zero-retrain transfer (partial)** — airline / banking A2-swap, fixed-part-invariant (`grep "if field"=0`, zero retrain), A2-swap effort, retention (eval-item 5). Structurally distant fields (healthcare) and the full ⑤ column remain [EST]; this upgrades transfer from premise to *partially measured*.
6. **[≈\$40–100] \$/req token-capture fix + the knee sweep** — turn on litellm cost/token so \$5.5 \$ becomes measured; sweep size vs total cost to locate the knee / fleet mix (eval-items 6/8).

Total ≈ \$300–800, within budget, prioritized so the backbone (1–2) lands first. Items that would exceed budget or require frontier delegation (full master matrix; 235B; full multi-field) stay [EST].

8. Discussion

The result reframes the deployment question from model size to **lever allocation under cost**. Two asymmetries do the work: capability is scale-retired *in a definite order* (so a recede-able capability-scaffold buys the small-model the dimensions scale would), and compliance is *scale-invariant* (so a guarantee-scaffold is kept even at frontier). Training a model to tolerate load is training it to do the deterministic controller's job; the thesis routes that work to determinism by default and to learning only for the

residual — a residual that is **scale-dependent** (empty at 32B operand, non-empty at 7B), which is precisely why the cost knee, not a fixed model size, is the right object.

9. Limitations

- **Scale range.** Measured 7B–32B; 72B/235B/frontier are [EST] (§7 item 2). The scale-invariance claim is a flat-trend + priors, strengthened by the 72B point once run.
- **Compliance metric.** The strong "scale-invariant" form depends on the per-opportunity g2 rate (§5.3 caveat, §7 item 1); absent that, we claim the *differential* (g1 collapses, g2 persists).
- **Observational vs controlled.** §5.2 correlations are size-confounded; only the controlled L_interf probe is causal; L_branch/L_len controlled curves and multi-turn (L_state/L_contra) probes are [EST].
- **Cost numbers.** \$/req is token-estimated (order-of-magnitude robust); the fleet assumes a decidable router not yet built; the human/Palantir comparisons are external baselines.
- **Transfer.** Zero-retrain A2-swap is the design premise; cross-field validation is [EST/partial] (§7 item 5) — and the "small + schema-swap → unseen domain" mechanism is itself a variant of schema-guided DST / ToolLLM (we claim the scale-invariance map, not the transfer mechanism, which belongs to Papers 2–3).
- **Single bench for compliance.** F3/F4 on τ^2 -retail; cross-domain/cross-bench is [EST].

10. Conclusion

Scaling a tool-use agent buys capability — it retires load dimensions in a definite order and lifts task pass — but it does **not** buy compliance: policy-semantic violations are flat across 7B→32B while pass climbs, and a deterministic gate removes them at every scale. The right object is therefore not the model size but the **capability×scale×lever×cost map**: a small base, a capability-scaffold that recedes as scale grows, a scale-invariant guarantee-scaffold, authored A2, and learning reserved for the scale-dependent residual — assembled at the total-cost knee. This is why even frontier systems should use deterministic scaffold and A2, and it grounds the companion papers that generate the A2 (Paper 2) and learn the path-selection residual (Paper 3).

[SETTLED] cells are measured (frontier-API-free); [EST] cells are the §7 funded plan. Bibliography (≈ 229 entries, deduplicated across the project corpus) follows; [unverified]/future-dated entries to be re-checked before submission.

Paper 1 — References (229 total, deduped across deep-research + litreview/relwork sources)

Collapsed 7 cross-source duplicate pairs (same arXiv id), merging the relationship notes of each: LATS (2310.04406), RAP (2305.14992), ReST-MCTS* (2406.03816), Math-Shepherd (2312.08935), PAL (2211.10435), ToolLLM (2307.16789), Voyager (2305.16291). Starting totals 89 (deep-research) + 147 (litreview) = 236; after dedup = 229.

1. Foundations — transformer expressivity & bounded-depth, procedural vs conceptual semantics, semantic automata, notationality & representation-cost

1. Kim & Suzuki (2024). *Transformers need intermediate-step supervision to learn parity in one step (ICLR25 Oral)*. arXiv:2410.08633. — [transformer-expressivity] intermediate-step loss yields proven efficiency separation; grounds derivation supervision.
2. Feng et al. (2023). *Towards Revealing the Mystery behind Chain of Thought*. arXiv:2305.15408 (NeurIPS23). — [transformer-expressivity] no-CoT const-depth = AC0/TC0; CoT evaluates size-T boolean circuit.
3. Feng et al. (2024). *Chain-of-Thought expressivity (follow-up)*. arXiv:2402.12875 (ICLR24). — [transformer-expressivity] CoT escapes TC0; supports inductive intermediate-emit prescription.
4. Merrill & Sabharwal (2022). *The Parallelism Tradeoff: Limits of Log-Precision Transformers*. arXiv:2207.00729. — [transformer-expressivity] log-precision forward pass = uniform TC0; bounded serial depth.

5. Bhattamishra et al. (2022). *On the difficulty of learning Boolean functions / low-sensitivity bias*. arXiv:2211.12316 (ACL23). — [transformer-expressivity] flat-AND easy, parity collapses; ICL accuracy vs boolean complexity $r=-0.88$.
6. Wang et al. (2024). *Boolean-function sensitivity in LLMs*. arXiv:2412.02823. — [transformer-expressivity] low-sensitivity bias corroborates flat-AND vs high-sensitivity collapse.
7. Abbe et al. (2024). *Globality barrier and inductive scratchpad*. arXiv:2406.06467 (NeurIPS24). — [transformer-expressivity] globality barrier; only inductive/structured scratchpad breaks it.
8. He et al. (2025). *Looped Locate-and-Replace (bottom-up reduction)*. arXiv:2512.02677. — [transformer-expressivity] bottom-up reduction mitigates OOD depth-decay; inference-time loop. [unverified — future-dated preprint]
9. Lakretz et al. (2021). *Depth generalization as a distinct failure axis*. arXiv:2101.02258. — [transformer-expressivity] depth generalization fails separately from length (PCC -0.92). [unverified — GPT-2-scale]
10. Dziri et al. (2023). *Faith and Fate: Limits of Transformers on Compositionality*. arXiv:2305.18654 (NeurIPS23). — [transformer-expressivity] transformers = linearized subgraph matching; error $\rightarrow 1$ with complexity; supports decomposition. (cross-link: §2 scale limits)
11. Jackson, Lee, Servidio & Wan (2008). *Learning random monotone DNF*. RANDOM08. — [transformer-expressivity] OR/disjunctive structure recovery is the core difficulty; classical PAC theory.
12. Zhou et al. (2023). *What Algorithms can Transformers Learn? (RASP-L length generalization)*. arXiv:2310.16028. — [transformer-expressivity] length-generalization criterion; flagged as killed/untrusted as a predictor. [unverified — claims killed in adversarial check]
13. Yehudai, Amsel & Bruna (2025). *CoT trades depth for sequential sub-result tokens*. arXiv:2503.01544 (NeurIPS25). — [transformer-expressivity] 2-layer transformer solves Boolean-formula CRQ via CoT depth $\leftrightarrow n$ exchange. [unverified — future-dated]
14. (RoPE bound authors) (2025). *Expressivity bound of RoPE transformers*. arXiv:2411.07602 (EMNLP25). — [transformer-expressivity] if $TC \neq NC1$, const-depth transformer cannot evaluate BFVP/arith-formula.
15. Malach (2019). *The role of local correlation in learning (gate-label)*. arXiv:1910.11923 (JMLR22). — [transformer-expressivity] gate-label local correlation needed for learnability.
16. Buss (1987). *The Boolean formula value problem is in ALOGTIME (BFVP NC1-complete)*. — [transformer-expressivity] BFVP=NC1-complete; theoretical ceiling for single-pass evaluation.
17. Dehghani et al. (2018). *Universal Transformers*. arXiv:1807.03819. — [transformer-expressivity] recurrence is Turing-complete (under assumptions); serial depth = iterations.
18. Giannou et al. (2023). *Looped Transformers as Programmable Computers*. arXiv:2301.13196. — [transformer-expressivity] looped 13-layer = programmable computer; constructive existence proof.
19. Fan et al. (2025). *Looping for length/depth generalization (n-RASP-L)*. arXiv:2409.15647 (ICLR25). — [transformer-expressivity] adaptive-step looping improves length/depth generalization with step supervision.
20. Geiping et al. (2025). *Huginn-3.5B: latent recurrent-depth reasoning*. arXiv:2502.05171. — [transformer-expressivity] from-scratch latent recurrence; retrofit-incompatible; latent < CoT. [unverified — future-dated]
21. Lu et al. (2025). *Probing latent CoT*. arXiv:2507.02199. — [transformer-expressivity] interpretable latent CoT nearly absent; latent $\blacksquare$ explicit CoT. [unverified — future-dated]
22. (TRM/recurrence authors) (2026). *Tiny Recursive Model / UT+ACT comparison*. arXiv:2604.21999. — [transformer-expressivity] recursive-depth TRM 87.4% vs UT+ACT 6-8% on Sudoku. [unverified — future-dated]
23. (recurrence-generalization authors) (2025). *Recurrence $\neq$ generalization alone*. arXiv:2510.04871. — [transformer-expressivity] recurrence needs correct training to generalize. [unverified — future-dated]
24. (additional depth-recurrence source) (2025). *Depth-recurrence internalization variant*. arXiv:2509.25239. — [transformer-expressivity] depth-recurrence source (primary list). [unverified — future-dated]
25. Li et al. (2024). *CoT enables serial / P-complete computation*. — [transformer-expressivity] CoT adds serial steps per token, enabling unbounded depth (P-complete); time-in-LLM payment.
26. van Benthem (1986). *Essays in Logical Semantics (semantic automata)*. — [semantic-automata] quantifiers encode automata complexity classes; "most" needs pushdown; depth measure foundation.

27. Barwise & Cooper (1981). *Generalized Quantifiers and Natural Language*. Linguistics and Philosophy. — [semantic-automata] generalized quantifier theory; basis for procedure-typed quantifier analysis.
28. Goodman (1968). *Languages of Art (notationality)*. — [procedural-semantics] notationality = disjointness + finite differentiation for unambiguous machine manipulation.
29. Blakemore / Wilson & Sperber (1987-2011). *Relevance Theory: procedural vs conceptual meaning*. — [procedural-semantics] linguistic items encode concepts (denotation) or procedures (inference routes); maps denotation vs procedure split.
30. Groenendijk, Stokhof, Veltman / Heim (1990s). *Dynamic semantics (meaning as context-change potential)*. — [procedural-semantics] meaning = information-state update instruction; denotation/procedure coexist.
31. Iverson (1979). *Notation as a Tool of Thought*. CACM (Turing Award lecture). — [procedural-semantics] notation embedding algorithm; depth-of-notation lineage. (cross-link: §12 classics)
32. Zhang & Norman (1995). *A representational analysis of numeration systems*. Cognition. — [procedural-semantics] representation sets cost profile not computability; Arabic vs Roman = internal vs external step tradeoff. (cross-link: §12 classics)
33. Bennett (1988). *Logical Depth and Physical Complexity*. — [procedural-semantics] logical depth = compute-time to unfold compact symbol; grounds d(e) notation-depth measure. (cross-link: §12 classics)
34. Wittgenstein (1953). *Philosophical Investigations (meaning as use)*. — [procedural-semantics] meaning-as-use captured by shallow association; LLM strength on use/sense. (cross-link: §12 classics)
35. Frege (1892). *Über Sinn und Bedeutung (sense vs reference)*. — [procedural-semantics] sense (grasp=association) vs reference (compute=procedure); LLM strong on sense, weak on reference-compute. (cross-link: §12 classics)
36. Brandom (1994/2000). *Material vs formal inference*. — [procedural-semantics] material (immediate) vs formal (computed) inference; LLM strong on material, stalls on formal. (cross-link: §12 classics)
37. Harris (1954) / Firth (1957). *Distributional hypothesis ("you shall know a word by the company it keeps")*. — [procedural-semantics] meaning as distributional geometry; low-dimensional manifold learnable by gradient descent.
38. Feldman (2013). *The neural binding problem(s) / ~100-step rule*. — [cognitive-architecture] brain ~100-step serial limit; language evolved for bounded-depth parallel cognition; grounds shallow-parallel thesis.
39. Clark & Chalmers (1998). *The Extended Mind*. Analysis. — [cognitive-architecture] humans offload deep procedures to external serial tools; LLM=language core, algorithm=external tool.

2. Scale limits & genuine model limits — compositional/reasoning collapse, hallucination inevitability & abstention, floor measurement, metacognition/self-evaluation

40. Xu, Jain & Kankanhalli (2024). *Hallucination is Inevitable: An Innate Limitation of LLMs*. arXiv:2401.11817. — [hallucination-abstention] formal (diagonalization) limit independent of scale; precision=1 unbuyable.
41. Kalai, Nachum, Vempala & Zhang (2025). *Why Language Models Hallucinate*. arXiv:2509.04664 (OpenAI). — [hallucination-abstention] hallucination = binary-classification errors + guessing-incentives; not a scale problem.
42. Mirzadeh et al. (2024). *GSM-Symbolic*. arXiv:2410.05229 (Apple). — [hallucination-abstention] single distractor clause drops SOTA up to 65%; no genuine logical reasoning.
43. Valmeekam, Stechly & Kambhampati (2024). *PlanBench / o1 evaluation*. arXiv:2409.13373. — [hallucination-abstention] o1 plain 97.8% but obfuscated 52.8%, 20-40 steps 23.6%; symbolic planner 100%.
44. (constraint-fabrication authors) (2025). *Reasoning models fabricate constraints*. arXiv:2505.12151. — [hallucination-abstention] reasoning models hallucinate absent constraints (67-94% of false errors). [unverified — future-dated]
45. (Reject-option survey authors) (2021). *Machine Learning with a Reject Option (survey)*. arXiv:2107.11277. — [hallucination-abstention] (h,r) predictor+rejector; 3 classes separated/dependent/integrated.
46. (Don't-Hallucinate-Abstain authors) (2024). *Don't Hallucinate, Abstain*. arXiv:2402.00367 (ACL 2024). — [hallucination-abstention] abstention externalized via multi-LLM collaboration (because self-reflection/held-out fails);

supports non-introspection.

47. Huang et al. (n.d.). *Large Language Models Cannot Self-Correct Reasoning Yet*. (ICLR). — [small-model-reasoning] internal self-correction degrades reasoning, external verify needed.
48. Zhang et al. (2024). *Small models need strong external verifiers*. ACL 2024. — [small-model-reasoning] small models fail self-verify on memorization-heavy substeps.
49. (Authors n/a) (2025). *T1: external-verifier scaling for small-model reasoning*. (2025). — [small-model-reasoning] small models need strong external verifier.
50. Zhou et al. (2023). *Least-to-Most Prompting Enables Complex Reasoning in LLMs*. ICLR 2023. — [small-model-reasoning] SCAN 16%→≥99%; decomposition not scale-gated.
51. Hsieh et al. (2023). *Distilling Step-by-Step*. ACL 2023. — [small-model-reasoning] 770M T5 beats few-shot 540B PaLM with 80% less data.
52. Chen et al. (2022). *Program of Thoughts (PoT): Disentangling Computation from Reasoning*. arXiv:2211.12588 / TMLR 2023. — [floor-measurement] program + deterministic interpreter offload, ~12% over CoT. (cross-link: §3 offload)
53. Joren et al. (2025). *Sufficient Context: A New Lens on Retrieval-Augmented Generation*. ICLR 2025. — [floor-measurement] context-sufficiency autorater; small=reasoning-limited, large=info-limited. (cross-link: §11 benchmarks)
54. Traub et al. (2024). *Overcoming Common Flaws in Selective Classification Evaluation (AUGRC)*. arXiv:2407.01032. — [metacognition] generalized risk-coverage curve; measurement discipline against single-point gaming. (cross-link: §12 classics)
55. (calibration-in-agents authors) (2026). *Verbalized confidence near-random in tau2-bench*. arXiv:2602.05073. — [metacognition] verbalized confidence/NLL/entropy near-random (AUROC 0.47-0.69); confidence inflates with trajectory length. [unverified — future-dated, pilot-scale]
56. (agentic-overconfidence authors) (2026). *Agentic overconfidence*. arXiv:2602.06948. — [metacognition] reinforces verbalized overconfidence finding in agents. [unverified — future-dated]
57. (tool-miscalibration authors) (2026). *Tool-call miscalibration*. arXiv:2601.07264. — [metacognition] tool-call miscalibration corroborating non-introspection. [unverified — future-dated]
58. (When2Call authors). *When2Call: learning when to call tools*. — [metacognition] when-to-call is learned (RPO■SFT); tool-necessity linearly decodable from hidden state (AUROC 0.89-0.96). (cross-link: §5 routing)
59. (Ask-or-Assume authors) (2026). *Ask-or-Assume: underspecification detection*. arXiv:2603.26233. — [metacognition] separates underspecification detection into a distinct agent; detector is an LLM judge. [unverified — future-dated]
60. (SAGE-Agent authors) (2025). *SAGE-Agent: structured belief state + EVPI stopping*. arXiv:2511.08798. — [metacognition] non-introspective structured belief + deterministic EVPI stopping; hand-specified Bayesian, not learned. [unverified — future-dated]

3. Neurosymbolic & offload (LLM proposes / deterministic engine executes) — PAL, LLM-Modulo, formalizers, RLVR-with-checker, neurosymbolic framing

61. Gao et al. (2022). *PAL: Program-Aided Language Models*. arXiv:2211.10435 / ICML 2023. — [offload] LLM decomposes / deterministic interpreter executes; PAL+Codex beats PaLM-540B+CoT by +15% on GSM8K; small+offload>540B; our division of labor exactly. (merged dr#83 + lit#96)
62. Kambhampati et al. (2024). *LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks*. arXiv:2402.01817 (ICML24). — [neurosymbolic] LLM proposer + external model-based verifier; 1:1 with our division of labor.
63. (LRM-Modulo authors) (2024). *LRM-Modulo: o1 with external verifier*. arXiv:2410.02162. — [neurosymbolic] o1 has zero correctness guarantee; soundness from external verifier; determinism orthogonal to capability.
64. (LLM-as-formalizer authors). *LLM-as-formalizer: NL→PDDL scaling*. — [offload] NL→PDDL formalizer beats direct planner (100 blocks 100% vs 20%); but loses 15/24 on simple problems (offload not always optimal).
65. (formalizer-erosion authors) (2024). *Erosion of formalizer advantage in top reasoning models*. arXiv:2412.09879. — [neurosymbolic] formalizer edge partially eroded on simple-PDDL (o3-mini/R1); messy-NL extension open. [unverified —

flagged open]

66. Marcus (2020). *The Next Decade in AI: Four Steps Towards Robust AI*. arXiv:2002.06177. — [neurosymbolic] hybrid knowledge-driven reasoning path for robust AI; anti scaling-only camp.
67. Garcez & Lamb (2020). *Neurosymbolic AI: The 3rd Wave*. arXiv:2012.05876 (AI Review 2023). — [neurosymbolic] motivation = trust/safety/interpretability/accountability; anchor for auditability.
68. Kautz (2022). *The Third AI Summer (Engelmore Lecture)*. AI Magazine 43(1):105-125. — [neurosymbolic] 6-type neurosymbolic taxonomy; positions our Neuro|Symbolic cooperative package.
69. LeCun (2022). *A Path Towards Autonomous Machine Intelligence*. OpenReview BZ5a1r-kVsf. — [neurosymbolic] modular world-model architecture; structure even in deep-learning camp.
70. Trinh et al. (2024). *Solving olympiad geometry without human demonstrations (AlphaGeometry)*. Nature 625. — [neurosymbolic] neural LM proposer + symbolic deduction engine; IMO geometry 25/30; frontier is hybrid.
71. DeepMind (2024). *AlphaProof / IMO 2024 silver*. DeepMind 2024. — [neurosymbolic] proofs RL'd over Lean formal verifier; reward = deterministic verifier (RLVR's V).
72. Peer & Stabinger (2025). *ATA: Neuro-Symbolic Autonomous Trustworthy Agents*. arXiv:2510.16381. — [neurosymbolic] LLM=NL→formal KB, symbolic engine decides; thesis-framing nearest rival; KB rebuilt per domain.
73. Evans et al. (2018). *Can Neural Networks Understand Logical Entailment? (PossibleWorldNets)*. arXiv:1802.08535 (ICLR18). — [neurosymbolic] tree-structured nets exploit logic syntax; semantic enumeration best; from-scratch only.
74. Bowman et al. (2014). *Recursive neural networks for logical semantics*. arXiv:1406.1827. — [neurosymbolic] tree-structured nets learn logical relations; not LLM/transfer.
75. Chowdhury & Caragea (2021). *Modeling Hierarchical Structures with CRvNN*. ICML21 (chowdhury21a). — [neurosymbolic] CRvNN extrapolates ListOps len≤100→900-1000 at 96-98%; recursive bias.
76. (Beam Tree RvNN authors) (2023). *Beam Tree Recursive Cells*. arXiv:2305.19999 (ICML23). — [neurosymbolic] recursive nets IID 99.4%; documents unseen arg-count (fan-in) generalization failure.
77. (RIR authors) (2023). *Recursion in Recursion*. arXiv:2311.04449 (NeurIPS23). — [neurosymbolic] recursive-structure models extrapolate depth/length where vanilla transformers fail.
78. Nye et al. (2019). *Learning to Infer Program Sketches (SketchAdapt)*. arXiv:1902.06349. — [neurosymbolic] learned sketch + symbolic hole-fill; boundary is learned (our delta axis). [unverified — PDF render failed, reconfirm]
79. Lin et al. (2022). *Description Logic TBox/ABox (terminology vs assertion separation)*. — [neurosymbolic] TBox/ABox term separation adopted as tool; their symbolic is hand-written, ours learned.
80. (Intermediate Languages authors) (2025). *Intermediate Languages Matter*. arXiv:2502.17216. — [neurosymbolic] NL→formal IR choice is first-order decision variable; single-shot ProntoQA/ProofWriter. (cross-link: §7 NL→formal)
81. Qiu, Ye, Gao, Zou, Chen, Gui, Huang, Xue, Qiu, Zhao (2025). *Blueprint First, Model Second: A Framework for Deterministic LLM Workflow*. arXiv:2508.02721. — [offload-boundary] deterministic blueprint backbone, LLM only where judgment needed. [quantified results unverified]
82. (Authors n/a) (2025). *From REST to MCP: An Empirical Study of API Wrapping and Automated Server Generation for LLM Agents (AutoMCP)*. arXiv:2507.16044. — [offload-boundary] spec→server compilation 76%→94.2% with auto-repair, cuts glue.
83. (PayPal authors) (2025). *A Declarative Language for Building And Orchestrating LLM-Powered Agent Workflows*. arXiv:2512.19769. — [cost-tradeoff] declarative DSL, 76% faster modifications, config not code. [self-reported]
84. (Authors n/a) (2025). *Agentic AI: A Comprehensive Survey of Architectures, Applications, and Future Directions*. arXiv:2510.25445. — [offload-boundary] symbolic/hybrid dominate safety-critical; generate-then-validate pattern.
85. (Authors n/a) (2020). *Robotic Process Automation — A Systematic Literature Review and Assessment Framework*. arXiv:2012.11951. — [cost-tradeoff] RPA low build cost but fragile/costly maintenance (non-vendor anchor). (cross-link: §10 cost/TCO)
86. (Authors n/a) (2026). *Automatic End-to-End Data Integration using Large Language Models*. arXiv:2603.10547. — [offload-boundary] LLM auto-generates schema/value mappings, replaces glue. [unverified, future-dated]

87. (Authors n/a) (2025). *LRASGen: LLM-based RESTful API Specification Generation*. arXiv:2504.16833. — [offload-boundary] LLM generates OpenAPI specs, cuts config-authoring cost.
88. (Authors n/a) (2024). *AI-powered software testing tools: A systematic review and empirical assessment of their features and limitations*. arXiv:2409.00411. — [offload-boundary] honest counterweight on self-healing locator limits.
89. (Authors n/a) (2026). *SAGAI-MID: A Generative AI-Driven Middleware for Dynamic Runtime Interoperability*. arXiv:2603.28731. — [offload-boundary] LLM as self-healing runtime integration layer. [unverified, future-dated]
90. (Authors n/a) (2024). *GRAM: Generative Retrieval Augmented Matching of Data Schemas*. arXiv:2406.01876. — [offload-boundary] generative schema matching ~88.7% vs 75.3% prior.
91. (Authors n/a) (n.d.). *Matchmaker: Self-Improving Large Language Model Programs for Schema Matching*. OpenReview:18E2ZooCte. — [offload-boundary] zero-shot self-improving schema matching, confidence-gated.
92. (Authors n/a) (n.d.). *A Neuro-Symbolic Framework for Deterministic Reliability in AI-Assisted Structural Engineering: The SYNAPSE Architecture*. MDPI 2075-5309/16/3/534. — [offload-boundary] deterministic safety-critical math, 94% accuracy <2s.
93. ZenML (2025). *What 1,200 Production Deployments Reveal About LLMOps in 2025*. zenml.io blog. — [offload-boundary] safety logic moved out of prompts into infrastructure.
94. Anthropic (n.d.). *Building Effective AI Agents*. anthropic.com/research. — [offload-boundary] deterministic workflows vs flexible agents, prefer simplest.
95. Stack Overflow Blog (2025). *Reliability for Unreliable LLMs*. blog (2025-06-30). — [offload-boundary] determinism from acceptance/validation; gate state/money/trust.
96. Google Developers Blog (n.d.). *Production-Ready AI Agents: 5 Lessons from Refactoring a Monolith*. blog. — [formalize-granularity] shift NL contract to runtime-validated typed object.
97. Google ADK (n.d.). *OpenAPI tools*. google.github.io/adk-docs. — [offload-boundary] auto-generate tools from OpenAPI; endpoint change = spec edit.
98. Microsoft Learn (n.d.). *Declarative agent architecture (Copilot extensibility)*. MS Learn docs. — [cost-tradeoff] config-not-code; OpenAPI-bound, no multistep loops.
99. Microsoft Azure AI Foundry (n.d.). *Three Tiers of Agentic AI — and When to Use None of Them*. vendor. — [offload-boundary] deterministic supervisor + agentic specialists.
100. deepset (n.d.). *AI Agents and Deterministic Workflows: A Spectrum, Not a Binary Choice*. vendor. — [offload-boundary] partition by criticality, not task sophistication.
101. Deepchecks (n.d.). *How Prompt Updates Drive Most Incidents*. vendor. — [cost-tradeoff] prompt surface itself a high change-cost liability.
102. Praetorian (n.d.). *Deterministic AI Orchestration: A Platform Architecture for Autonomous Development*. blog. — [offload-boundary] mission-critical needs deterministic, auditable execution.
103. samchon (n.d.). *samchon/openapi — OpenAPI converters + LLM function-calling schema composer*. GitHub. — [formalize-granularity] validation feedback raises tool-call success 70%→98%.

4. Cognitive architectures × LLM — SOAR, CoALA, NL2GenSym, Bootstrapping, LLM-ACTR, MERLIN2

-
104. (NL2GenSym authors) (2025). *NL2GenSym: NL→SOAR production via LLM*. arXiv:2510.09355. — [cognitive-architecture] LLM generates NL→SOAR rules, execution-grounded Generator-Critic; nearest neighbor; small+framework>large.
 105. Wray, Kirk & Laird (2025). *Applying Cognitive Design Patterns to General LLM Agents*. arXiv:2505.07087 (AGI 2025). — [cognitive-architecture] impasse/subgoal, propose-select-reconsider as canonical; LLM self-reflection distrust; authority for SOAR framing.
 106. Sumers, Yao, Narasimhan & Griffiths (2024). *Cognitive Architectures for Language Agents (CoALA)*. arXiv:2309.02427 (TMLR). — [cognitive-architecture] umbrella framework; LLM=probabilistic production; memory/action/decision axes; we

are an instance.

107. Wu, Oltramari, Francis, Giles & Ritter (2024). *Cognitive LLMs / LLM-ACTR*. arXiv:2408.09176. — [cognitive-architecture] ACT-R policy baked into LoRA weights; opposite direction; no transfer/abstain.
108. Zhu & Simmons (2024). *Bootstrapping Cognitive Agents with a Large Language Model*. arXiv:2403.00810 (AAAI 2024). — [cognitive-architecture] LLM bootstraps SOAR-like productions, symbolic verifies; deterministic-first/LLM-fallback, 50-100× token reduction.
109. González-Santamarta et al. (2023). *LLM in Cognitive Architecture (MERLIN2 robot)*. arXiv:2309.14945. — [cognitive-architecture] GBNF grammar-constrained decoding NL→PDDL, symbolic planner verifies; LLM=generate/symbolic=execute.
110. (NL2CA authors) (2025). *NL2CA: NL→LTL→ACT-R production compilation*. arXiv:2512.18189. — [cognitive-architecture] fine-tuned Qwen3-0.6B NL→LTL→Critic Tree→pyactr production, fully automatic; direct prior for NL→A2 generator. [unverified — future-dated] (cross-link: §7 NL→formal)
111. Jones, Wray & Laird (2026). *No-compliant-action as structural event (AAAI-26)*. AAAI 2026 (paper 41081). — [cognitive-architecture] structural no-compliant-action event; flagged as verification-failed, close-read needed. [unverified — verification failed in adversarial check]

5. Learned routing, deferral & small-model orchestration (cost) — ToolOrchestra, TRUST, RTR/A2FM/xRouter/When2Call/RITE, cascading/deferral (ReDAct, Unified Routing/Cascading)

-
112. (ToolOrchestra authors) (2025). *ToolOrchestra*. arXiv:2511.21689. — [cost-routing] 8B RL: tau2 80.2%@10.3¢ > GPT-5 77.7%@31.3¢; most dangerous rival, preempts "small>large" headline. [unverified — future-dated]
 113. (TRUST authors) (2026). *TRUST: 4-way Direct/Tool/Ask/Unable routing*. arXiv:2606.06976. — [cost-routing] 4B>30B, Claude-Sonnet-4 parity; pure RL+perplexity, introspective; small>large co-preemptor, differentiate by method. [unverified — future-dated]
 114. (RTR authors). *RTR: joint model+strategy routing*. — [cost-routing] learned joint routing with OOD transfer, 71.7% token reduction; learned routing is established.
 115. (A2FM authors). *A2FM: instant/reasoning/agentic mode RL routing*. — [cost-routing] RL routing across modes, 45% cost reduction; learned routing established.
 116. (xRouter authors). *xRouter: answer-vs-delegate RL routing*. — [cost-routing] RL answer-vs-delegate routing; learned routing established.
 117. (ARM2 authors). *ARM2: NL/code/vision format RL selection*. — [cost-routing] RL format selection with code offload; learned routing established.
 118. (ToolkenGPT authors). *ToolkenGPT: tool calls as token prediction*. — [cost-routing] tool invocation as token prediction; learned routing established.
 119. (RITE authors). *RITE: math-only RL training transfers cross-domain SOTA*. — [cost-routing] decomposition/tool-routing is a learned, transferable skill; supports our C8 transfer claim.
 120. (TGRL authors) (2025). *TGRL: domain-invariant planning via RL*. arXiv:2510.11184. — [cost-routing] tool-use habit transfer via opaque RL; no basis/closure claim. (cross-link: §9 transfer)
 121. (ReDAct authors) (2026). *ReDAct: Uncertainty-Aware Deferral for LLM Agents*. arXiv:2604.07036. — [cost-routing] calibrated-threshold defer small→large on ALFWorld/MiniGrid. [unverified — future-dated]
 122. (Routing-Cascading authors) (2024). *Unified Routing and Cascading for LLMs*. arXiv:2410.10347. — [cost-routing] cascade routing; learned quality estimator is the key factor; model-selection deferral.
 123. (cascade-error authors) (2025). *Modular agent cascade errors (quantified)*. arXiv:2503.13657. — [cost-routing] modular-agent cascade errors are real and quantified.
 124. (Self-Healing authors) (2026). *Self-Healing Agentic Orchestrators*. arXiv:2606.01416. — [cost-routing] deterministic monitor→diagnose→recover→verify loop; fault-injection 98.8%. [unverified — future-dated]

125. (AgentDebug authors) (2025). *AgentDebug / Where LLM Agents Fail*. arXiv:2509.25370. — [cost-routing] AgentErrorTaxonomy + attribution/repair; +24%/+17%/up to 26% on ReAct benchmarks.
126. Shinn et al. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. arXiv:2303.11366 / NeurIPS 2023. — [learned-routing] external episodic memory, frozen policy, fast trial gains.
127. Zhao et al. (2024). *ExpeL: LLM Agents Are Experiential Learners*. arXiv:2308.10144 / AAAI 2024. — [learned-routing] distills traces into NL insights, cross-task transfer.
128. Wang et al. (2023). *Voyager: An Open-Ended Embodied Agent with LLMs*. arXiv:2305.16291. — [learned-routing] skill library indexed by NL, frozen GPT-4, compose skills; open (infinite, domain-specific) skill library = structural opposite of our closed basis. (merged dr#38 + lit#112)
129. Wang, Mao, Fried, Neubig (2024). *Agent Workflow Memory (AWM)*. arXiv:2409.07429. — [learned-routing] induces reusable abstract workflows, +24.6%/+51.1% relative.
130. (Authors n/a) (2025). *Adaptation of Agentic AI: A Survey of Post-Training, Memory, and Skills*. arXiv:2512.16301. — [learned-routing] parametric vs external memory vs skill libraries. [unverified, future-dated]
131. (Authors n/a) (2026). *Memory for Autonomous LLM Agents: Mechanisms, Evaluation, and Emerging Frontiers*. arXiv:2603.07670. — [learned-routing] distilled abstract skills > raw retrieval banks. [unverified, future-dated]
132. (Authors n/a) (2025). *A Benchmark for Procedural Memory Retrieval in Language Agents*. arXiv:2511.21730. — [learned-routing] evaluates retrieving past procedures from experience DB. [unverified, future-dated]
133. (Authors n/a) (2024). *Enhancing Decision-Making for LLM Agents via Step-Level Q-Value Models*. arXiv:2409.09345. — [learned-routing] external Q-store guides selection without retraining policy.
134. (Authors n/a) (2025). *DG-PRM: Dynamic and Generalizable Process Reward Modeling*. arXiv:2507.17849 / ACL 2025. — [learned-routing] external reward-tree dynamically selected, OOD-generalizable; swappable store.
135. (Authors n/a) (2026). *ToolPRMBench: Evaluating and Advancing PRMs for Tool-using Agents*. arXiv:2601.12294. — [learned-routing] extends step-PRMs to tool-use, checklist sub-goal rewards. [unverified, future-dated]
136. Arumugam et al. (2025). *Toward Efficient Exploration by Large Language Model Agents*. arXiv:2504.20997. — [learned-routing] PSRL via LLMs, retains provable Bayesian regret guarantees.
137. Xia et al. (2024). *Beyond Numeric Rewards: In-Context Dueling Bandits with LLM Agents (LEAD)*. arXiv:2407.01887. — [learned-routing] LEAD inherits regret guarantees from external bandit, not LLM.
138. (Authors n/a) (2024). *Which LLM to Play? Convergence-Aware Online Model Selection with Time-Increasing Bandits (TI-UCB)*. arXiv:2403.07213. — [learned-routing] non-stationary success-rate prioritizer with regret analysis.
139. (Authors n/a) (2025). *A Multi-Agent Conversational Bandit Approach to Online Evaluation and Selection of User-Aligned LLM Responses*. arXiv:2501.01849. — [learned-routing] provably-improving external selector over LLM candidates.
140. (Authors n/a) (2025). *QLASS: Boosting Language Agent Inference via Q-Guided Stepwise Search*. arXiv:2502.02584. — [learned-routing] Q-guided stepwise search (raw hit). [unverified]
141. (Authors n/a) (2025). *AgentPRM: Process Reward Models for LLM Agents via Step-Wise Promise and Progress*. arXiv:2511.08325. — [learned-routing] PRM capturing decision interdependence (raw hit). [unverified, future-dated]
142. (Authors n/a) (2025). *TDRM: Smooth Reward Models with Temporal Difference*. arXiv:2509.15110. — [learned-routing] smoothed reward models (raw hit). [unverified]
143. (Authors n/a) (n.d.). *Principle Process Reward (PPR)*. OpenReview (no arXiv id). — [learned-routing] unifies step assessment + outcome verification (raw hit). [unverified]
144. (Authors n/a) (2025). *Agent KB: Leveraging Cross-Domain Experience for Agentic Problem Solving*. arXiv:2507.06229. — [learned-routing] cross-domain experience reuse (raw hit). [unverified]
145. (Authors n/a) (2026). *ProcMEM: Learning Reusable Procedural Memory via Non-Parametric PPO*. arXiv:2602.01869. — [learned-routing] reusable procedural memory (raw hit). [unverified, future-dated]
146. (Authors n/a) (2025). *LEGOMem: Modular Procedural Memory for Multi-agent LLM Systems*. arXiv:2510.04851. — [learned-routing] modular procedural memory (raw hit). [unverified]

147. (Authors n/a) (2025). *How Memory Management Impacts LLM Agents: experience-following behavior*. arXiv:2505.16067. — [learned-routing] memory management / experience-following (raw hit). [unverified]

6. Guarantees, shielding & safety enforcement — safe-RL shielding, CBF/safety-filters, ShieldAgent, AgentSpec, Formal-LLM, solver-aided policy compliance

148. Alshiekh et al. (2018). *Safe Reinforcement Learning via Shielding*. AAAI 2018. — [shielding-safety] LTL spec→deterministic automaton→safety game→shield; archetype of propose-then-shield.
149. Jansen et al. (2018). *Safe RL via Probabilistic Shields*. arXiv:1807.06096. — [shielding-safety] probabilistic relaxation of shielding; variant lineage.
150. Wabersich & Zeilinger (2021). *Predictive safety filter (pCBF)*. Automatica 2021; arXiv:2105.10241. — [shielding-safety] modular safety filter appended to any learned controller; proposer-independent.
151. Hewing/Wabersich et al. (2023). *The Safety Filter: A Unified View*. Annual Reviews 2023. — [shielding-safety] CBF/predictive-filter/HJ-reachability unified survey; paradigm maturity.
152. (ShieldAgent authors) (2025). *ShieldAgent: guardrail agent from policy rules*. arXiv:2503.22738. — [shielding-safety] policy→verifiable rules→probabilistic rule circuits; nearest neighbor, probabilistic not deterministic. [unverified — abs read only]
153. (AgentSpec authors) (2025). *AgentSpec: runtime constraint DSL*. arXiv:2503.18666. — [shielding-safety] trigger/predicate/enforcement DSL, deterministic but hand-written per-domain spec. (cross-link: §7 NL→formal)
154. (Formal-LLM authors) (2024). *Formal-LLM: automaton-constrained generation*. arXiv:2402.00798. — [shielding-safety] developer-written automata supervise generation; generation-constraint variant, no transfer.
155. (ProbGuard authors) (2025). *ProbGuard: probabilistic runtime monitoring*. arXiv:2508.00500. — [shielding-safety] probabilistic runtime monitoring; one-line variant. [unverified — abs only]
156. Winston, Winston & Just (2026). *Solver-Aided Verification of Policy Compliance*. arXiv:2603.20449. — [shielding-safety] NL policy→SMT-LIB→Z3 runtime gate on tau-bench; gate-leg nearest rival; human-translated, airline only. [unverified — future-dated]

7. NL→formal / spec generation & constrained decoding — NL2CA, Prose2Policy, AgentSpec, StepFun-Formalizer, FormalAlign, VeriEquivBench, intermediate-languages, CRANE / Let-Me-Speak-Freely

157. Guo et al. / IRNet (2019). *Towards Complex Text-to-SQL in Cross-Domain Database with Intermediate Representation (IRNet/SemQL)*. arXiv:1905.08205 / P19-1444. — [formalize-granularity] schema-link→SemQL→deterministic SQL; reference-emit, cross-domain transfer.
158. Dong & Lapata (2018). *Coarse-to-Fine Decoding for Neural Semantic Parsing*. ACL P18-1068. — [formalize-granularity] value-free sketch first then fill, shared across examples.
159. (Authors n/a) (2019). *Sketch-based semantic parsing*. arXiv:1909.00574. — [formalize-granularity] sketch-then-fill; value-filling is itself learned.
160. (Authors n/a) (2021). *PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding*. EMNLP 2021.emnlp-main.779. — [constrained-decoding] T5-3B 68-71%→75.1%, execution error 12%→2%.
161. (Authors n/a) (2020). *RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL*. arXiv:1911.04942. — [formalize-granularity] schema-as-input relation-aware self-attention, zero-shot schema transfer.
162. (Authors n/a) (2017). *Execution-Guided Decoding for Text-to-SQL*. arXiv:1807.03100. — [constrained-decoding] deterministic execution engine filters faulty programs, WikiSQL 83.8%.
163. (Authors n/a) (2025). *Grammar-Constrained Decoding as Parser (form-not-meaning)*. ACL 2025.acl-industry.34. — [constrained-decoding] grammar guarantees form not meaning; CFG ceiling, needs solver. [semantic-improvement claim REFUTED]

164. Tam et al. (2024). *Let Me Speak Freely? A Study on the Impact of Format Restrictions on Performance of LLMs*. arXiv:2408.02442 / EMNLP 2024 Industry. — [constrained-decoding] schema-JSON drops GSM8K ~26–73%; two-stage recovers.
165. (Authors n/a) (2025). *CRANE: Reasoning with Constrained LLM Generation*. arXiv:2502.09061 / ICML 2025. — [constrained-decoding] grammar augmentation + alternating constrained/free, +10pp GSM-sym/FOLIO.
166. Ye et al. (2024). *Guided Decoding distribution distortion (GAD)*. NeurIPS 2024. — [constrained-decoding] greedy token-masking + renormalization distorts distribution (KL-bias).
167. (Authors n/a) (n.d.). *JSONSchemaBench (in-schema reasoning-field-first)*. (benchmark). — [constrained-decoding] free-text reasoning field inside schema first recovers reasoning.
168. Kim, Poiroux & Bosselut (2026). *Do LLMs Game Formalization?*. arXiv:2604.19459 (EPFL, ICLR-2026 VerifAI workshop). — [hallucination-abstention] NL→Lean4: R1 premise-mistranslation undetectable, GPT-5 axiom-fabrication detectable; supports silent-residual delta. [unverified — future-dated]

8. Search, planning, tree-evaluation & process-reward internalization — ToT/RAP/LATS/ToolChain*, Searchformer/TS-LLM, Math-Shepherd/Let's-Verify/ReST-MCTS/OmegaPRM, loop-vs-CoT, tree-eval learnability, path/plan selection

169. Yao et al. (2023). *Tree of Thoughts (ToT)*. arXiv:2305.10601 (NeurIPS23). — [search-internalization] external tree search; CoT 4%→74%; analogical motivation (we remove external search).
170. Hao et al. (2023). *Reasoning with Language Model is Planning with World Model (RAP)*. arXiv:2305.14992 / EMNLP 2023. — [search-internalization] LLM as world model + policy, process-reward MCTS; LLaMA-33B+RAP > GPT4-CoT +33%; external search to be internalized. (merged dr#32 + lit#26)
171. Zhou et al. (2024). *Language Agent Tree Search (LATS)*. arXiv:2310.04406 / ICML 2024. — [search-internalization] external MCTS controller + LLM self-eval value, no weight updates; 92.7% HumanEval; analogical. (merged dr#31 + lit#27)
172. Besta et al. (2024). *Graph of Thoughts (GoT)*. AAAI24 (29720). — [search-internalization] graph beats tree on accuracy and cost (>31% reduction); analogical motivation.
173. (DPTS authors) (2025). *Dynamic Parallel Tree Search (DPTS)*. arXiv:2502.16235 (ACL25). — [search-internalization] external tree-search efficiency; analogical. [unverified — future-dated]
174. Zhuang et al. (2024). *ToolChain*: Efficient Action Space Navigation with A* Search*. arXiv:2310.13227 / ICLR 2024. — [search-internalization] external A* cost/heuristic over API-call decision tree.
175. (Authors n/a) (2025). *Unifying Tree Search Algorithm and Reward Design for LLM Reasoning: A Survey*. arXiv:2510.09988. — [search-internalization] taxonomizes tree-search families vs reward designs.
176. (Authors n/a) (2024). *Scaling of Search and Learning: A Roadmap to Reproduce o1*. arXiv:2412.14135. — [search-internalization] search+learning roadmap (raw hit). [unverified]
177. (Authors n/a) (2026). *ToolTree: Dual-Feedback MCTS + Bidirectional Pruning*. arXiv:2603.12740. — [search-internalization] tool-planning tree search (raw hit). [unverified, future-dated]
178. Lehnert et al. (2024). *Searchformer: Beyond A with search-dynamics bootstrapping*. arXiv:2402.14083 (TMLR, Meta). — [search-internalization] keystone: A trace distillation + shorter-trace bootstrapping; teacher-exceeding generalization.
179. (Searchformer-critique authors) (2025). *Critique of Searchformer mechanism*. arXiv:2505.13775. — [search-internalization] critiques the mechanism (not the numbers) of trace-distillation. [unverified — future-dated]
180. Feng et al. (2024). *TS-LLM: Tree-Search with learned value function*. arXiv:2309.17179 (ICML24). — [search-internalization] learned value-function distillation reaches depth-64; adaptation for depth risk.
181. Zhang et al. (2024). *ReST-MCTS*: LLM Self-Training via Process Reward Guided Tree Search*. arXiv:2406.03816 (NeurIPS24). — [search-internalization] external search guidance plus per-step process reward auto-inferred from final answer; internalization via self-training; recipe for our verifier. (merged dr#35 + lit#33)

182. Wang et al. (2024). *Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations*. arXiv:2312.08935 / ACL 2024. — [small-model-reasoning] auto step-PRM, Mistral-7B GSM8K 89.1%; step labels from final gate-answer; external + internalize modes. (merged dr#47 + lit#34)
183. Lightman et al. (2023). *Let's Verify Step by Step*. arXiv:2305.20050 / OpenAI; ICLR 2024. — [small-model-reasoning] PRM>ORM, MATH 78%, PRM800K; external verifier ranks candidates.
184. Luo et al. (2024). *OmegaPRM / Improve Mathematical Reasoning by Automated Process Supervision*. arXiv:2406.06592 / Google DeepMind. — [small-model-reasoning] MCTS+binary search locates first error, scales PRM labels.
185. Tian et al. (2024). *AlphaLLM (imagination-searching-criticizing)*. arXiv:2404.12253 (NeurIPS24). — [search-internalization] search-trace self-train > ReST^EM/Self-Rewarding; expert-iteration recipe.
186. Guan et al. (2025). *rStar-Math: 7B self-evolution via MCTS*. arXiv:2501.04519 (ICML25). — [search-internalization] 7B self-evolution without superior teacher; 58.8→90 MATH number is killed/cite-forbidden. [unverified — specific number killed]
187. Song et al. (2024). *Trial and Error: Exploration-Based Trajectory Optimization (ETO)*. arXiv:2403.02502 / ACL 2024. — [search-internalization] whole-trajectory DPO internalizes success>failure into weights.
188. Putta et al. (2024). *Agent Q: Advanced Reasoning and Learning for Autonomous AI Agents*. arXiv:2408.07199. — [search-internalization] MCTS + self-critique + off-policy DPO. [figures unverified]
189. (additional internalization source) (2024). *Self-train / expert-iteration variant*. arXiv:2404.03683. — [search-internalization] internalization recipe (primary source list).
190. (additional internalization source) (2024). *Trace/value internalization variant*. arXiv:2405.14838. — [search-internalization] internalization recipe (primary source list).
191. (additional internalization source) (2024). *Trace/search internalization variant*. arXiv:2407.06023. — [search-internalization] internalization recipe (primary source list).
192. Xu & Sato (2025). *To CoT or To Loop? A formal comparison*. arXiv:2505.19245. — [transformer-expressivity] deterministic DAG/tree evaluation: loop $\propto$ depth, CoT $\propto$ size (formal separation). [unverified — future-dated]
193. (RELAY authors) (2025). *RELAY: loop-iteration $\leftrightarrow$ CoT-step alignment*. arXiv:2502.08482. — [search-internalization] recurrence as trace generator; distill looped traces into AR model via SFT. [unverified — future-dated]
194. (Authors n/a) (2025). *A Survey of Process Reward Models: From Outcome Signals to Process Supervisions*. arXiv:2510.08049. — [small-model-reasoning] PRM survey (raw hit). [unverified]
195. (Authors n/a) (n.d.). *Toward Large Reasoning Models: reinforced reasoning with LLMs*. PMC12546433. — [small-model-reasoning] reasoning-model survey (secondary). [unverified]

9. Transfer — schema-guided dialogue & tool function-calling — SGD, D3ST, STAR, Description-Driven, ToolLLM, Tool-Doc, TGRL

-
196. Rastogi et al. (2020). *Schema-Guided Dialogue (SGD)*. — [benchmarks] finite acts + schema for unseen-service transfer; DST not tool-use planning; no closure proof.
 197. (D3ST authors) (2022). *D3ST / Description-Driven TOD*. arXiv:2201.08904. — [benchmarks] schema reading (NL description→slot); pure neural DST; no closure basis.
 198. (STAR authors) (2020). *STAR: schema-guided task-oriented dialogue with flowcharts*. arXiv:2010.11853. — [benchmarks] finite acts, explicitly non-closed; flowchart=data; exact opposite of our rule-as-abstraction.
 199. (ToolLLM authors) (2023). *ToolLLM: Facilitating LLMs to Master 16000+ Real-World APIs*. arXiv:2307.16789. — [benchmarks] open tool set (16k, infinite); cross-API OOD; pure LLM; NL→API tool-plan setting; preempts cross-bench transfer. (merged dr#75 + lit#128)
 200. (Tool-Doc authors) (2023). *Tool documentation enables zero-shot tool usage*. arXiv:2308.00675. — [benchmarks] tool-doc-driven tool use; FC transfer mechanism.

201. (Schema Augmentation authors) (2024). *Schema Augmentation for zero-shot transfer*. arXiv:2411.00150. — [benchmarks] schema augmentation for unseen-domain transfer; concrete prior for ABox-swap.
202. Patil et al. (2023). *Gorilla: Large Language Model Connected with Massive APIs*. arXiv:2305.15334. — [formalize-granularity] NL→API calling setting (sourced, verification claim did not survive).
203. Li et al. (2023). *API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs*. arXiv:2304.08244. — [formalize-granularity] NL→API benchmark (sourced, verification claim did not survive).

10. Cost / TCO / regulatory / scaling stance — deterministic-vs-learned TCO, Sutton/Brooks/Chollet, Palantir, EU AI Act / SR 11-7 / SR 26-2 / EU MDR / MiFID, GDPR

204. Sutton (2019). *The Bitter Lesson*. incompleteideas.net/IncIdeas/BitterLesson.html. — [bitter-lesson] general methods leveraging computation win; search AND learning; targets injected know-how, not spec-enforcement.
205. Brooks (2019). *A Better Lesson*. rodnebrooks.com/a-better-lesson/. — [bitter-lesson] human architecture design relocated not removed; total-cost framing = our small+gate leg.
206. Chollet (2019). *On the Measure of Intelligence*. arXiv:1911.01547. — [bitter-lesson] unlimited priors/data "buy" skill, masking generalization; justifies zero-retrain transfer measurement.
207. Karpathy (2025). *On Sutton's Bitter Lesson (X / interview reaction)*. X, 2025-10. — [bitter-lesson] bitter lesson as "biblical text" in frontier circles; non-academic, contested. [unverified — non-academic]
208. Sutton & Patel (2025). *Richard Sutton interview ("LLMs are a dead end") + follow-up*. Dwarkesh Patel podcast 2025-09. — [bitter-lesson] Sutton: LLMs inject human knowledge, not bitter-lesson-pilled; undercuts attack's minor premise.
209. European Union (2024). *EU AI Act, Regulation (EU) 2024/1689*. EUR-Lex ELI reg/2024/1689. — [regulatory-determinism] mandates traceability/logging/oversight; "deterministic"=0 occurrences; Art.15(4) presumes adaptive systems.
210. Federal Reserve (2011). *SR 11-7: Guidance on Model Risk Management*. Fed SR Letter 2011. — [regulatory-determinism] presumes statistical/uncertain models; requires validation/effective-challenge, not determinism.
211. Fed/OCC/FDIC (2026). *SR 26-2: Model Risk Management (supersedes SR 11-7/21-8)*. Fed SR Letter SR2602; OCC Bulletin 2026-13. — [regulatory-determinism] excludes deterministic rule-based processes from "model"; genAI/agent out of scope.
212. European Union (2017). *EU MDR 2017/745, Annex I §17.1*. EUR-Lex CELEX:32017R0745. — [regulatory-determinism] only explicit "repeatability" clause; medical-device, "in line with intended use" qualified.
213. FDA (2019). *AI/ML-Based SaMD Discussion Paper ("locked algorithm")*. FDA 2019-04. — [regulatory-determinism] historical "locked algorithm" definition; nonbinding; trajectory moving away from locked.
214. FDA/Health Canada/MHRA (2021). *Good Machine Learning Practice (GMLP) 10 Guiding Principles*. gov.uk MHRA publication. — [regulatory-determinism] "deterministic/reproducible/repeatable"=0; robustness + real-world monitoring.
215. European Union (2016). *GDPR Article 22 (automated decision-making)*. gdpr-info.eu. — [regulatory-determinism] safeguards = human intervention + contest; no determinism requirement.
216. European Commission (2017). *MiFID II RTS 6, Commission Delegated Regulation (EU) 2017/589*. EUR-Lex CELEX:32017R0589. — [regulatory-determinism] behave-as-intended testing; no determinism wording, (b)-light burden. [unverified — snippet, reconfirm before citing]
217. Snell et al. (2024). *Scaling LLM Test-Time Compute Optimally....* arXiv:2408.03314. — [cost-tradeoff] external search-vs-verifier beats bigger policy at fixed compute, 14× larger.
218. Ampersand (n.d.). *The Integration Debt Trap: Why Building Integrations In-House Breaks Down at Scale*. withampersand.com. — [cost-tradeoff] 160–480 hr/yr, \$50k–\$150k/yr per integration.
219. Airbyte (n.d.). *How to Build API Integrations That Don't Break*. airbyte.com. — [cost-tradeoff] one provider change propagates across all tenants.
220. Duvo (n.d.). *Why Every RPA Project Breaks (And How Agentic AI Fixes It)*. blog.duvo.ai. — [cost-tradeoff] 30–50% RPA failure, presentation-layer coupling.

221. Truto (n.d.). *Building Integrations In-House and Other Horror Stories*. vendor. — [cost-tradeoff] integrations silently stop syncing until customer notices.
222. denlava / DEV Community (2023). *Reducing API Test Brittleness: Strategies Against Minor Schema Changes*. DEV (2023-04-03). — [cost-tradeoff] hardcoded fields +70% test maintenance; schema-aware cuts 50–70%.
223. Ampersand / Towards Data Science (n.d.). *From Vibe Coding to Spec-Driven Development*. TDS. — [cost-tradeoff] spec as source of truth makes approach robust.
224. (Author n/a) (2026). *Spec-driven development case study with Claude Code*. blog (2026-05-12). — [cost-tradeoff] spec authority discipline. [partial identifier]

11. Benchmarks — tau2, SOPBench, SOP-Bench, TaskBench, CFB, AppWorld, TravelPlanner, NESTful, NATURAL PLAN, Sufficient Context

225. Nangia & Bowman (2018). *ListOps: A Diagnostic Dataset for Latent Tree Learning*. arXiv:1804.06028 (NAACL18). — [benchmarks] nested-operator evaluation diagnostic; latent-tree structure discovery fails, not capacity.
226. Tay et al. (2021). *Long Range Arena (LRA)*. arXiv:2011.04006 (ICLR21). — [benchmarks] long-range benchmark including ListOps; structure-discovery scope.
227. (ORCHARD authors) (2021). *ORCHARD: nested-operator evaluation*. arXiv:2111.14034. — [benchmarks] nested-operator scope diagnostic; scope-only relevance.

12. Classics & adopted tools — Codd, Böhm-Jacopini, Libkin-Wong, Goodman, van Benthem, Wittgenstein/Frege/Brandom, Iverson, Bennett, Zhang-Norman, Koriat/Fleming/Rouy, Traub AUGRC, Tülu 3, InstructGPT/DPO/STaR/RAG

228. Codd (1970/1972). *Relational completeness of data base sublanguages (relational algebra)*. — [procedural-semantics] relational algebra as closed finite content-operation basis; data-flow generators.
229. Böhm & Jacopini (1966). *Flow diagrams, Turing machines and languages with only two formation rules*. CACM. — [procedural-semantics] structured-program theorem; closed finite flow basis (P1-P9) for tool-use planning.

Unverified / to-recheck before submission

The following entries carry an [unverified], future-dated, (Authors n/a), partial-identifier, or verification-failed flag and must be re-checked before submission:

- §1: 8, 9, 12, 13, 20, 21, 22, 23, 24
- §2: 44, 55, 56, 57, 59, 60
- §3: 65, 78, 81, 82 (Authors n/a), 83, 84 (Authors n/a), 85 (Authors n/a), 86, 87 (Authors n/a), 88 (Authors n/a), 89, 90 (Authors n/a), 91 (Authors n/a), 92 (Authors n/a)
- §4: 110, 111
- §5: 112, 113, 121, 124, 130 (Authors n/a), 131 (Authors n/a), 132 (Authors n/a), 133 (Authors n/a), 134 (Authors n/a), 135 (Authors n/a), 138 (Authors n/a), 139 (Authors n/a), 140 (Authors n/a), 141 (Authors n/a), 142 (Authors n/a), 143 (Authors n/a), 144 (Authors n/a), 145 (Authors n/a), 146 (Authors n/a), 147 (Authors n/a)
- §6: 152, 155, 156
- §7: 159 (Authors n/a), 160 (Authors n/a), 161 (Authors n/a), 162 (Authors n/a), 163 (Authors n/a), 165 (Authors n/a), 167 (Authors n/a), 168
- §8: 173, 176 (Authors n/a), 177, 179, 186, 188, 192, 193, 194 (Authors n/a), 195 (Authors n/a)
- §10: 207, 216, 224 (partial identifier)

Note: many (Authors n/a) / (Author n/a) entries reflect placeholder author fields carried verbatim from the source lists; these need an authoritative author attribution before final citation. All future-dated arXiv ids (2025–2606) are flagged as [unverified] per source.